第一阶段：向量加法指令执行过程

工大喵喵队 2026 年 11 月

一、分析对象

vadd.vv 是标准 RVV 的向量加法：

```
for i in 0 .. vl-1:
    vd[i] = vs2[i] + vs1[i]
```

我们选它做分析，是因为它不依赖自定义指令，可以先把香山的向量流水线跑通、看清。测试配置是 VL=16、SEW=8、LMUL=1，指令为 vadd.vv v3, v1, v2，PC 是 0x80000044，编码是 0x021101D7。

测试程序在 software/vadd_only.S：

```
vsetivli zero, 16, e8, m1, tu, mu
vle8.v   v1, (a0)
vle8.v   v2, (a1)
vadd.vv  v3, v1, v2
vse8.v   v3, (a2)
ret
```

反汇编里能看到目标指令：

```
80000038 <vadd_demo>:
  80000038: c0087057   vsetivli zero,16,e8,m1,tu,mu
  8000003c: 02050087   vle8.v   v1,(a0)
  80000040: 02058107   vle8.v   v2,(a1)
  80000044: 021101d7   vadd.vv  v3,v1,v2
  80000048: 020601a7   vse8.v   v3,(a2)
  8000004c: 8082       ret
```

二、测试程序和波形

vadd_main.c 准备 A/B 数组、调用 vadd_demo、逐字节比对结果。make vadd 跑 40 组功能测试，make vadd-wave 只跑 1 组，方便采波形。

默认的 emu 没有编译 FST 跟踪，我们单独构建了一个开 EMU_TRACE=fst 的 emu-fst，保留原来的 build/emu 不动。运行 vadd-wave.bin 时打开--dump-wave，得到 vadd-stage1.fst，大小 20,054,170 字节，SHA256：

```
b257a425a9ae0965f9cb205b1eb955d4b4d026dbe3e511acd6fc422a8d6454e8
```

这次运行的日志是：

```
Core 0: HIT GOOD TRAP at pc = 0x80000032
Core-0 instrCnt = 440, cycleCnt = 2,728, IPC = 0.161290
```

这次采波形用的是 vadd-wave（x0 trap 字），只能说明运行至结束、无 mismatch；a0 版 vadd 回归是 23,876 条指令、16,870 周期、a0=0，引用在 `05` 里。

整段程序只有 440 条指令，波形窗口比较干净。FST 太大，我们用 `tools/extract_vadd_wave.py` 把 2480–2565 的窗口裁出来，再用 `tools/report_vadd_wave.py` 做 34 项采样断言，生成 `vadd-focused.vcd` 和 `vadd-wave-evidence.json`。34 项断言都成立。

三、执行过程

执行过程如图 1 所示。

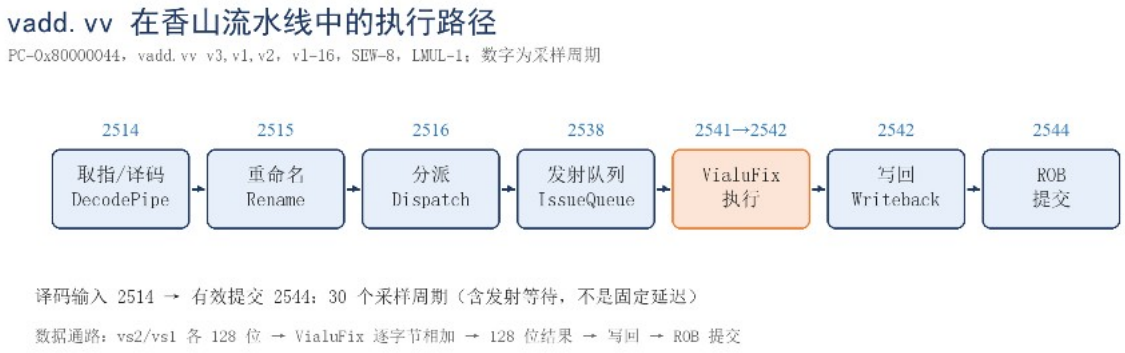


图 1 vadd.vv 在香山流水线中的执行路径

关键采样点整理如下：

采样值	阶段	看到的信号	说明
2514	译码输入	`decodePipeRenameModule.io_in_valid=1`、`ready=1`	PC=`0x80000044`，instr=`0x021101D7`
2515	译码输出/重命名	`io_out_valid=1`、`ready=1`	PC、instr 与输入一致
2516	分派	`dispatch.io_fromRename_0_valid=1`、`ready=1`	ROB=(flag=1, value=33)，pdest=51
2538	发射	`IssueQueueVfmaVialuFixVfalu.io_deqDelay_0_valid=1`、`ready=1`	ROB、pdest 与分派一致
2541	执行输入	`VialuFix.io_in_valid=1`、`ready=1`	`src_0=B`，`src_1=A`

采样值	阶段	看到的信号	说明
2542	执行输出	VialuFix.io_out_valid=1、ready=1、vecWen=1	ROB、pdest 一致，结果 C=A+B
2543	提交槽	commitValid_0=0	只能看到 PC，不算提交
2544	有效提交	isCommit=1、commitValid_0=1	debug_pc=0x8000000 44

2514 到 2515，译码到重命名的流水寄存器输入、输出都有效，PC 和指令字对得上，说明指令完成译码，进入重命名。

2516，分派接口 fromRename_0 有效，ROB 表项是(flag=1, value=33)，物理目的寄存器 pdest=51，也就是 v3。后面发射、写回、提交都靠这个 ROB 标签和 pdest 串起来。

2538，发射队列 deqDelay_0 有效，ROB 和 pdest 与分派一致，说明这条 uop 等到操作数就绪后被选中。

2541 到 2542，VialuFix 输入有效，两路 128 位源数据：

信号	值（高位到低位）	解码
src_0（B）	0x1e1c1a18161412100e0c0a0806040200	B=[0,2,4,…,30]
src_1（A）	0x0f0e0d0c0b0a09080706050403020100	A=[0,1,2,…,15]

2542 输出有效、vecWen=1、ROB/pdest 一致：

信号	值（高位到低位）	解码
res_data（C）	0x2d2a2724211e1b1815120f0c09060300	C=[0,3,6,…,45]

16 个元素全部满足 C[i]=A[i]+B[i]。硬件 src_0 对应汇编里的第二个源操作数 B，src_1 对应第一个源操作数 A，这是香山操作数端口的连接顺序。

2542 写回，vecWen=1；2543 提交槽 0 能看到目标 PC，但 commitValid_0=0，只是 speculative 可见；2544 isCommit=1、commitValid_0=1，vadd 正式提交。

从译码输入 2514 到有效提交 2544，共 30 个采样周期。这个数字包含发射等待和流水开销，不是 vadd 的固定 latency。

四、数据结果

16 个元素逐项列出来：

i	A[i]	B[i]	C[i]=A+B
0	0	0	0
1	1	2	3
2	2	4	6
3	3	6	9
4	4	8	12
5	5	10	15
6	6	12	18
7	7	14	21
8	8	16	24
9	9	18	27
10	10	20	30
11	11	22	33
12	12	24	36
13	13	26	39
14	14	28	42
15	15	30	45

这组数据 A≤15、B≤30、C≤45，没有超过 int8 的 127，所以不涉及溢出。这组数据只说明这一组算对了，不能当成所有 int8 组合都验证过。

五、时间说明

区间	采样间隔	含义
译码输入 2514 → 译码输出 2515	1	译码流水寄存器
分派 2516 → 发射 2538	22	发射队列等待/仲裁，不是固定延迟
执行输入 2541 → 输出 2542	1	VialuFix 组合逻辑/流水
输出 2542 → 提交 2544	2	写回广播、ROB 提交

区间	采样间隔	含义
译码输入 2514 → 提交 2544	30	完整前端到提交，含等待

FST 元数据里写的是 1 ps，但我们用的是普通`--dump-wave`，时间值来自`trap_event.cycleCnt`，所以横轴实际是周期计数，不是物理时间。

六、本阶段结果

· 在香山`MinimalConfig`仿真里抓到了`vadd.vv`的完整执行链：译码 2514、重命名 2515、分派 2516、发射 2538、执行 2541–2542、提交 2544。

· 16 个字节的 A/B/C 全部逐元素匹配，ROB 标签、pdest、vecWen 在发射/写回/提交之间保持一致。

· 波形时间值是周期计数，30 个采样周期包含等待，不能当固定延迟。

· 波形来自`MinimalConfig`，没有采`KunminghuV2MinimalConfig`的 FST；精简窗口也没有保留取指阶段信号，所以文档里不写“取指已核验”。